

# CSCI 264: Assembly Language

Fall 2026

Computer Science, College of Arts & Sciences, Widener University • 4 credits

---

**Instructor**

Dr. Michael West  
mjwest@widener.edu  
Office: Freedom 319

**Office Hours**

M–F 2:00–3:00 (Thursday on Zoom), and by appointment.

---

| Section | Component | Days & Time        | Room          |
|---------|-----------|--------------------|---------------|
| A       | Lecture   | MWF 10:00–10:50 AM | Kirkbride 236 |
| A       | Lab       | Tue 12:30–1:45 PM  | Freedom 303   |
| B       | Lecture   | MWF 11:00–11:50 AM | Kirkbride 232 |
| B       | Lab       | Tue 3:30–4:45 PM   | Freedom 303   |

---

## Course Description

---

The aim of this course is to illuminate the structure and behavior of computers. It covers arithmetic, logic, and stack operations, the general organization of a CPU, main memory, and peripheral systems. The structure of executable files, linking, interfacing assembler with a high-level language and the operating system, and protected instructions are also discussed. Supervised laboratory exercises are used to illustrate all the main topics of the course. Some familiarity with a high-level programming language is desirable.

## Prerequisites

None.

## Learning Outcomes

---

By the end of this course, students will be able to:

1. Convert among binary, hexadecimal, and decimal representations, and explain how integers, negative numbers, and floating-point values are encoded and where those encodings fail.
2. Read and write x86-64 assembly implementing arithmetic, conditionals, loops, function calls, recursion, arrays, and structs.
3. Write, compile, and debug non-trivial C programs that use pointers, dynamically allocated memory, structs, and multiple source files.
4. Describe the von Neumann model and trace the execution of an instruction through a processor.

## Required Materials

---

**Textbook.** Suzanne J. Matthews, Tia Newhall, and Kevin C. Webb, *Dive into Systems*, Version 1.2, 2020. Free online at [diveintosystems.org/book](https://diveintosystems.org/book). There is no cost and no code to purchase; a print edition exists but is not required.

**A note on assembly syntax.** This course uses AT&T syntax throughout, matching the textbook. Online references and older course materials frequently use Intel syntax instead, in which the operand order is reversed. If a code sample you find online does not look like what we do in class, check which syntax it is written in before concluding that one of us is wrong.

## Assessment

---

| Component       | Weight      |
|-----------------|-------------|
| Quizzes         | 10%         |
| Midterm Exam 1  | 15%         |
| Midterm Exam 2  | 15%         |
| Final Exam      | 20%         |
| Lab Assignments | 40%         |
| <b>Total</b>    | <b>100%</b> |

## Grading Scale

---

|    |        |    |       |    |          |
|----|--------|----|-------|----|----------|
| A  | 93–100 | B  | 83–86 | C  | 73–76    |
| A– | 90–92  | B– | 80–82 | C– | 70–72    |
| B+ | 87–89  | C+ | 77–79 | D  | 60–69    |
|    |        |    |       | F  | below 60 |

---

## Component Descriptions

**Quizzes.** There will be a weekly quiz, on Canvas, due at midnight on Fridays. Quizzes will be short knowledge checks, with infinite attempts.

**Exams.** Two midterms and a cumulative final. Exams are closed-book; students may bring a single, handwritten cheat sheet.

**Lab Assignments** Lab sections meet each Tuesday. Lab assignments will generally be hands on programming problems. Assignments will be introduced during lecture on Monday and due at midnight on Monday (one or two weeks later, depending on the complexity of the assignment). Assignments must be submitted on Canvas by the stated due date to receive full credit. **In order to receive a grade for a lab assignment you must demo to me in the lab.**

## Course Policies

---

### Attendance

There is no grade for attendance, however lecture and lab attendance is expected. The material is cumulative and moves quickly so an absence is expensive in a way it may not be in other courses. If you must miss a session, get the notes from a classmate before coming to office hours.

### Classroom Behavior

Feel free to leave the room at any time, but please try not to distract other students. You may eat, drink and be merry so long as it does not disrupt the lecture.

**No laptops during lecture.** I encourage you to take hand-written notes.

### Late Work

Late quizzes will receive 50% credit.

Lab Assignments submitted late on Canvas will lose 10% per day, up to 50%.

Demos will be graded based on the Canvas submission. My goal is to get every demo done during the lab session, but if we run out of time you will be expected to demo during office hours. Assignments must be demoed in the same week they are due or will be penalized 20%.

### Collaboration

Discussing concepts, debugging strategies, and the meaning of an assignment with classmates is encouraged. What you submit must be written by you. Concretely: talking at a whiteboard is fine; looking at another student's code file, or sharing yours, is not. If a classmate helps you solve a problem, name them in a comment at the top of your submission — this is never penalized.

### Generative AI

Generative AI tools – software that creates new text, images, computer code, audio, video, and other content – are widely available. Well-known examples include ChatGPT, Claude, Gemini, and Grammarly for text and DALL-E and Midjourney for images. This policy governs all such tools, including those released during our term together, in accordance with Widener University's academic integrity standards. You may use generative AI tools for assistance, subject to the conditions below.

**You make a good faith effort to complete the assignment on your own.** The goal of this course is to build an understanding of and skill with the technologies that we cover. If you simply outsource the work, you are robbing yourself of the experience that builds this understanding and skill.

**You own every line you submit.** Whatever produced it, the code is yours to explain, justify, and modify. A submission you cannot explain or alter is treated as a submission you did not write, regardless of whether it compiles and passes every test. The purpose of the lab demos is for me to test your **understanding** of the code you submit, not its correctness.

**Disclose what you used.** Include a short comment block at the top of your main source file naming any AI tool you used and what you used it for (one or two sentences is enough). Disclosure is never penalized and never lowers a grade. Using a tool without disclosing it, or misrepresenting what you used it for, is an academic integrity violation and will be handled under the university policy below.

**Exams are closed-book and tool-free.** Midterms and the final are taken without devices or assistance of any kind. Exams are 50% of your grade.

This is not permission to outsource the work; it is an acknowledgment that these tools exist, that you will use them in your career, and that the useful skill is knowing when they help.

Uses that tend to help: explaining a compiler error you do not recognize, clarifying what a library function does, reviewing code you have already written. Uses that tend to hurt: generating a function before you have attempted it, and asking for an explanation of code you are supposed to be reading yourself. If you are unsure whether a specific use is permitted, ask me. I will be more lenient when you ask for permission than when you ask for forgiveness.

### **Academic Integrity**

Refer to the University's [Academic Integrity Policy](#).

### **Accessibility and Accommodations**

Any student has the right to request reasonable accommodation of a disability under the Americans with Disabilities Act (ADA) and Section 504 of the Rehabilitation Act of 1973. Student Accessibility Services reviews documentation and student requests for accommodations.

I understand that disclosure of a disability and advocating for adequate accommodations can be complex. If you have any needs or requests which would improve your experience in my class, please feel free to ask.

### **Student Support and Well-Being**

Widener's Counseling and Psychological Services ([CAPS](#)) offers free, short-term counseling to help students manage stress, navigate life changes, and develop coping skills.

Widener offers [academic advising](#), [coaching](#) and [tutoring](#).

## Important Dates

---

Exam Dates are **tentative**.

| Date               | Event                   |
|--------------------|-------------------------|
| Tue, Sep 1         | Last day for add/drop   |
| Tue, Oct 6         | <b>Midterm Exam 1</b>   |
| Mon–Tue, Oct 19–20 | Fall Break no classes   |
| Fri, Nov 13        | Last day to withdraw    |
| Tue, Nov 17        | <b>Midterm Exam 2</b>   |
| Thu–Fri, Nov 26–27 | Thanksgiving no classes |
| Fri, Dec 4         | Last day of classes     |
| Mon–Fri, Dec 7–11  | <b>Final Exams</b>      |

## Course Schedule

---

This schedule is a rough plan; readings and topics may shift as the semester develops.

| Wk | Reading                | Topic                                                                                |
|----|------------------------|--------------------------------------------------------------------------------------|
| 1  | §1.1–1.2; §4.1–4.2     | Unix; compiling and running C; base conversion                                       |
| 2  | §1.3–1.5; §4.3–4.4     | C control flow, functions, arrays, and strings; signed binary                        |
| 3  | §4.5–4.6; §4.8.2–4.8.3 | Integer overflow; bitwise operators; IEEE 754 floating point                         |
| 4  | §2.1–2.9;              | Pointers; the heap and memory management in C                                        |
| 5  | §5.1–5.4               | The von Neumann architecture; logic gates and circuits                               |
| 6  | §5.5–5.9               | Building a processor; instruction execution; pipelining                              |
| 7  | Ch 6, §2.9.7; §7.1     | <b>Midterm Exam 1</b> Compiling C to Assembly; registers, operands, addressing modes |
| 8  | §7.1–7.2               | Assembly basics continued; common instructions                                       |
| 9  | §7.3                   | <i>Fall Break.</i> Arithmetic instructions                                           |
| 10 | §7.4                   | Condition codes; conditionals and loops in assembly                                  |
| 11 | §7.4–7.5               | Loops continued; functions and the calling convention                                |
| 12 | §7.5–7.6               | Stack frames; recursion                                                              |
| 13 | §7.7                   | <b>Midterm Exam 2 (in lab).</b> Arrays in assembly                                   |
| 14 | §7.8–7.10              | <i>Thanksgiving.</i> Matrices and structs in assembly; buffer overflows              |
| 15 | Ch 10; Ch 11           | Key assembly takeaways; the memory hierarchy and CPU caches                          |
| 16 | —                      | <b>Final Exam</b> — cumulative                                                       |